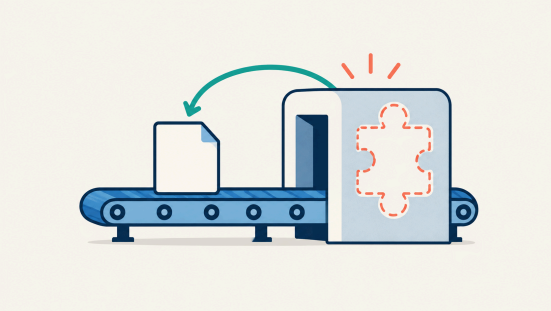
The Missing Test Suite

Where the bottleneck goes when code is cheap

Daniel Fridljand · danielfridljand.de

TNG Technology Consulting
4 December 2026



- About 20 minutes, then discussion.
- A project I took from discovery to a live shadow deployment: an LLM automation for first-level customer support.
- Not a talk about prompts or models. A talk about where the time went, and about the part that ran like a software factory: a coding agent and CI turning each decision into a branch, an eval case and a green build overnight.

Early in the project, we had to decide: workflow or agent?

Worth a talk of its own: Workflow or Agent.
Today we take it as fixed: we built a workflow.

- Early in the project, we had to decide how much of the work the model does and how much code does: one agent holding every tool, or a workflow where code decides the order of operations.
- That decision is worth a separate presentation, and there is a post on it.
- In this talk we take it as fixed that we decided on the workflow. Everything that follows, the loop and the overnight fixes, runs on top of that workflow.

Code is cheap now.
So what becomes the bottleneck?

On a support automation I built: the experts who could tell whether a reply was right.

- The question hanging over coding agents: once code is cheap, what is slow?
- My answer, from one project: the people who know the business. Not because they are slow, but because they are the only check the system has.
- The rest of the talk: why that is, the loop we built around it, and what I would do differently.

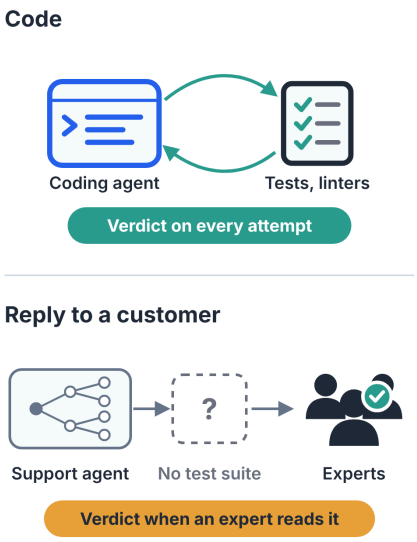
Why coding agents work

1. **Software is familiar.** Syntax, idioms and libraries are patterns the model has seen countless times in training.
2. **Every attempt gets a verdict.** Tests, linters and error messages tell the agent whether it worked, so the agentic loop runs without a human in it.

- Two reasons, and the second is the one that matters for this talk.
- The agent tries, the test suite answers, the agent tries again. Nobody has to read the code for the loop to make progress.
- Ask the room: how many of you let an agent run against a test suite unattended? That is the free verdict.

A reply to a customer has no test suite

- Code: the verdict comes **on every attempt**, for free.
- A reply: the verdict comes **when an expert reads it**.
- The model hasn't seen your escalation rules or your customers' records either.



- Neither condition holds for a business process.
- Records: a read tool, an endpoint the system can call, solves that.
- Rules: a read tool would cover them too, if anyone had written them down. Usually nobody has; they live in the heads of the people who handle the tickets.
- So the only check on a reply is a person who knows the business reading it. That is the missing test suite, and it is where the effort belongs.

The running example

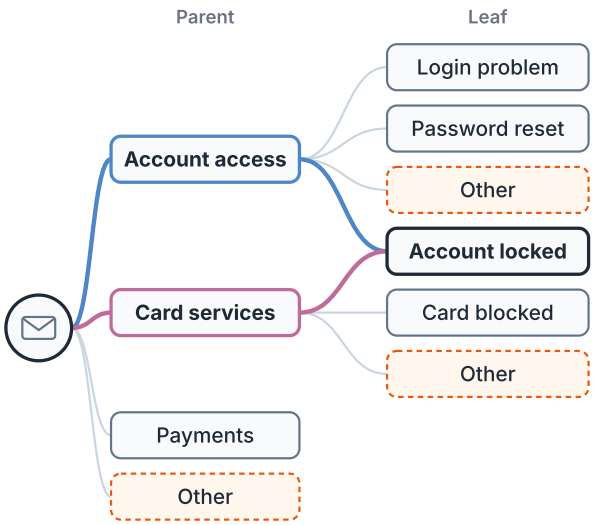
Customer · 2026-03-12 09:41

Hi, I'm Alex Carter and I bank with your Riverton branch. My account number is NSB01234 and I can't get in any more; it keeps saying the account is locked. Can you help me get back in?

Invented to protect the client; the project behind it is real.

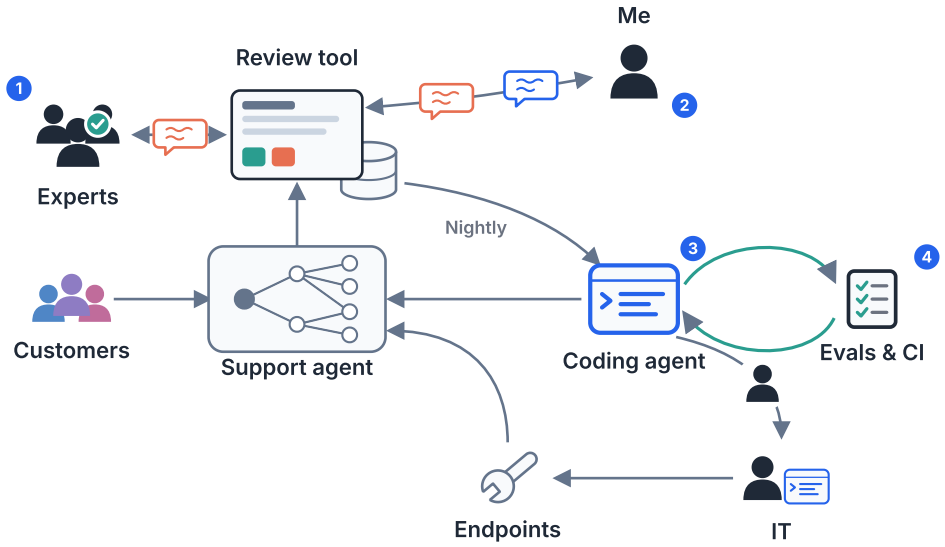
- Say it out loud: the bank, Alex and the account are invented. The project is a real customer support automation.
- First-level support: the messages that come in before anyone specialised looks at them.
- A locked account is the commonest kind of request, and simple on the surface. Keep Alex in mind; we follow this ticket through the whole loop.

The workflow: the model picks the request, code decides what happens



- The workflow's structure, briefly, because it is what the coding agent changes every night. More in Architecture of a First-Level Support Automation.
- A rooted DAG of intents, mostly two levels deep. One narrow LLM call picks the parent, a second picks the child.
- Every parent has an explicit "Other": coverage gaps land somewhere visible.
- Alex goes Account access → Account locked. Then code decides: look the customer up, check the flag, send the unlock link or escalate to security. The model reads; code decides the order.
- What matters here: the business logic lives in code, in small, separate places, so most expert feedback becomes a change in one place: a new intent or a new branch. That is what makes a fix cheap to specify and safe to hand to a coding agent.

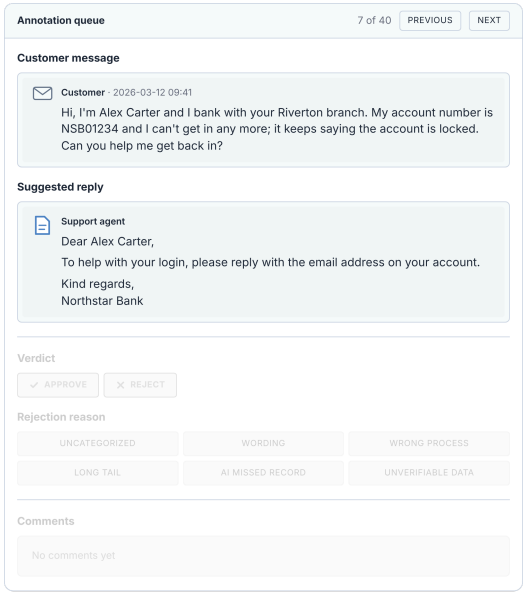
The whole system



- The map for the rest of the talk; each section highlights its part.
- Customers write in, the support agent drafts a reply, the draft lands in a review tool.
- The numbers are the loop: 1 experts grade, 2 I specify the fix, 3 a coding agent implements it overnight, 4 evals and CI check it.
- Bottom right: endpoints, built by the client's IT. Each new kind of answer needed one; IT worked from an OpenAPI spec I committed and usually delivered within a day.

Step 1: experts grade

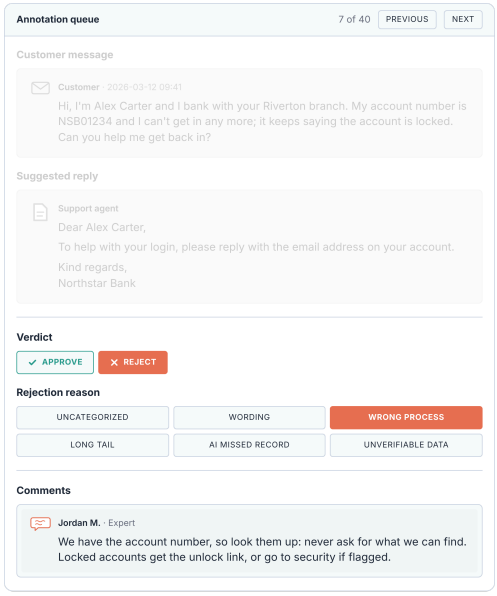
- The customer’s message and the proposed reply, nothing else.
- Shadow deployment: a draft for every real ticket, none sent.
- Graded when the expert picks up the ticket anyway.
- An approved draft is a reply they don’t have to write.



- The system drafted a reply to every real ticket; nothing was sent automatically. The experts were the support staff who answered those tickets anyway: happy with the draft, they paste it in; not happy, they write their own.
- A mockup, with the invented example.
- Before the tree had an Account locked intent, Alex’s message landed in “Other”, and the fallback agent wrote this: it asks Alex for the email address on the account.
- Ask the room: what’s wrong with it? (We already have the account number.)

One rejection, two rules nobody had written down

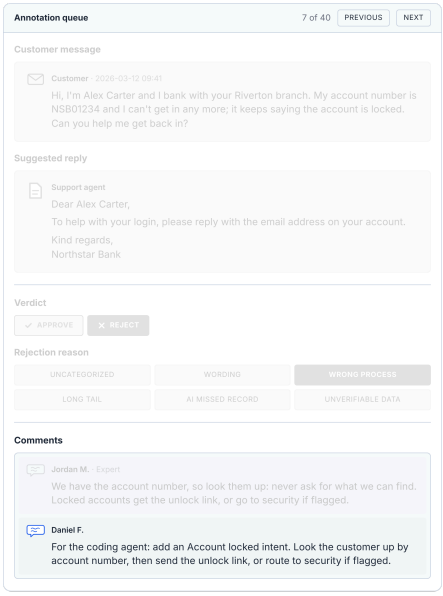
- **Never ask for what we can find.**
- **Locked accounts get the unlock link, or go to security if flagged.**
- Grading is the check a coding agent gets from its tests, and how the rules get written down.



- Verdict: reject. Reason: wrong process. And a comment.
- The people who know a business can rarely write its rules down in advance. Shown a draft, they see at a glance what is wrong with it.
- One bad reply brought out two rules. That is the best ratio you will get: you can’t interview your way to these.
- So grading does two jobs at once.

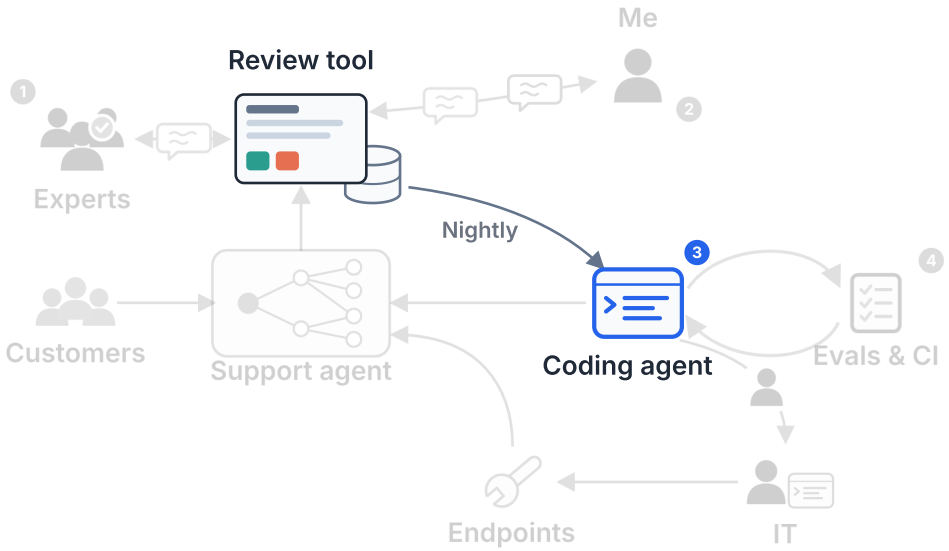
Step 2: I specify the fix

- In one sitting, for a batch of feedback.
- A plan for the coding agent next to the expert's comment: where the fix goes, what it does.
- Follow-up questions go in the tool, next to the ticket.



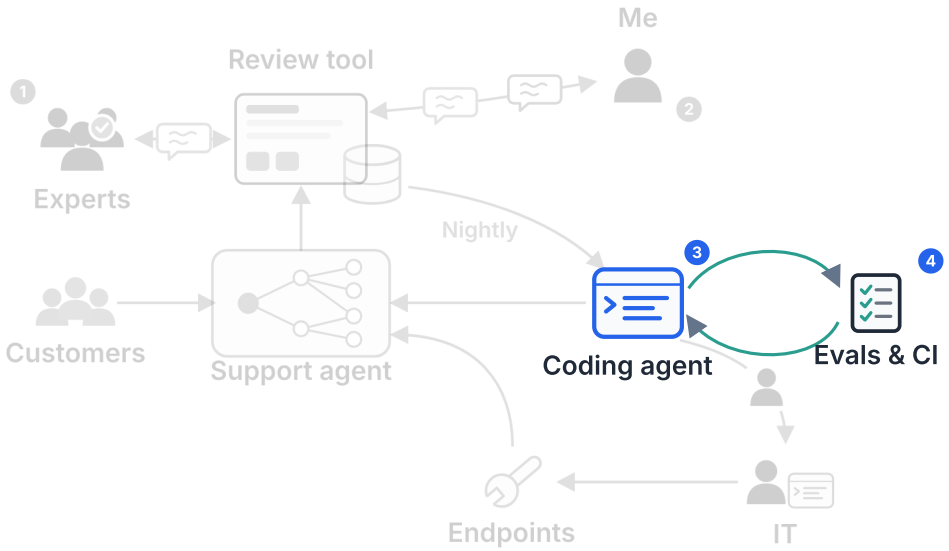
- I go through a batch of stored feedback in one sitting and add my comment next to each expert's: a high-level implementation plan for the coding agent.
- Where in the system the fix goes, and what it should do. Not the code.
- This comment is the factory's work order. Because most fixes land in one place, two sentences are usually enough.
- Nobody messaged anybody: comments stay with their ticket and wait until someone is ready. That is what made ten experts workable for one developer.

Step 3: the coding agent implements it overnight



- A nightly run picks up every ticket where I have specified a fix, all at once.
- Usually that means adding a new intent to the intent DAG, or a new branch to the rules behind one. For Alex's ticket: a new Account locked intent.
- With a coding agent, a fix I had specified was a branch, an eval case and a green build by the next morning, with little of my time.

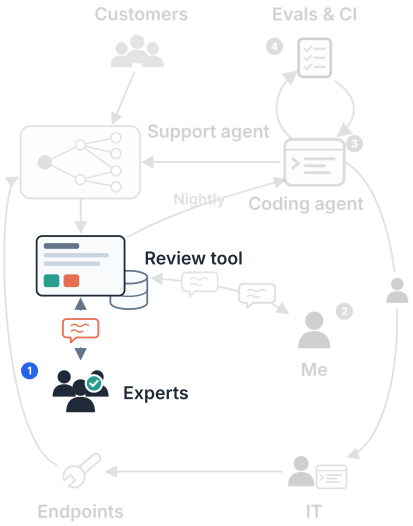
Step 4: evals and CI check it



- The agent adds the case to the eval set: here Alex's message and the reply it should get.
- CI re-runs the whole suite, so earlier fixes stay fixed.
- This step is an ordinary test suite, and it is the easy part: once a case is written down, the rest is ordinary engineering.
- Notice what happened: the expert's verdict became a test. The missing test suite gets written one rejected draft at a time.
- Eval runs go to Langfuse, one trace per case tied to a commit, and when a case fails, subagents triage it. More in Langfuse in Production.

The factory checks against my plan; only an expert checks against the business

- The eval case is written from **my** plan.
- A green build says the code does what I asked, not that I asked for the right thing.
- So I resubmit the rejected ticket, and an expert checks the new draft.



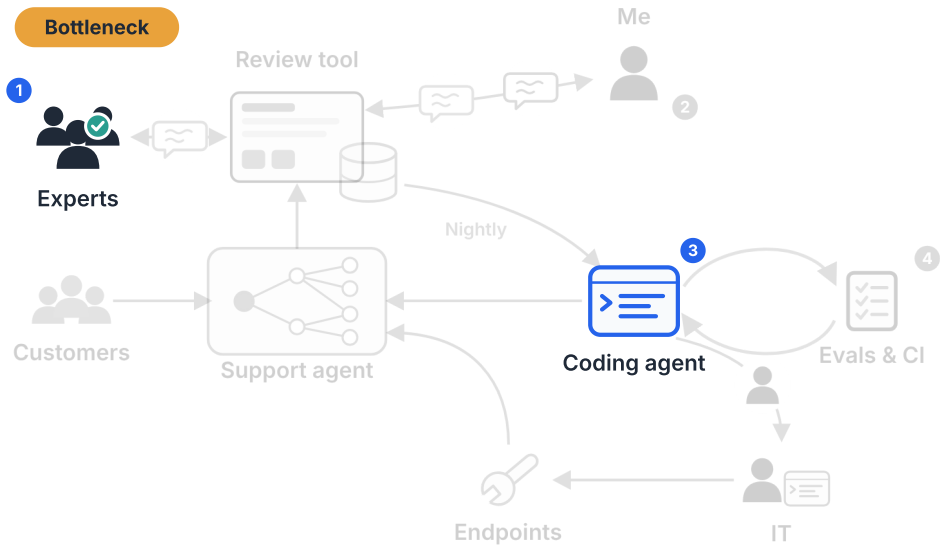
- Then the fix ships, and I resubmit the rejected ticket from the review tool: the support agent drafts a new reply, and the draft goes back into the shared queue.
- Whichever expert picks it up checks the fix on the ticket that exposed the error. The eval case can’t do that.
- A rejected ticket leaves the loop only when an expert approves it.
- This is the limit of the software factory here: everything it verifies, it verifies against a spec. For code, the spec is the tests. For a business process, the spec is in the experts’ heads.

Every system has a bottleneck.
Fixing it doesn’t remove it; it moves it.

Eliyahu Goldratt, theory of constraints

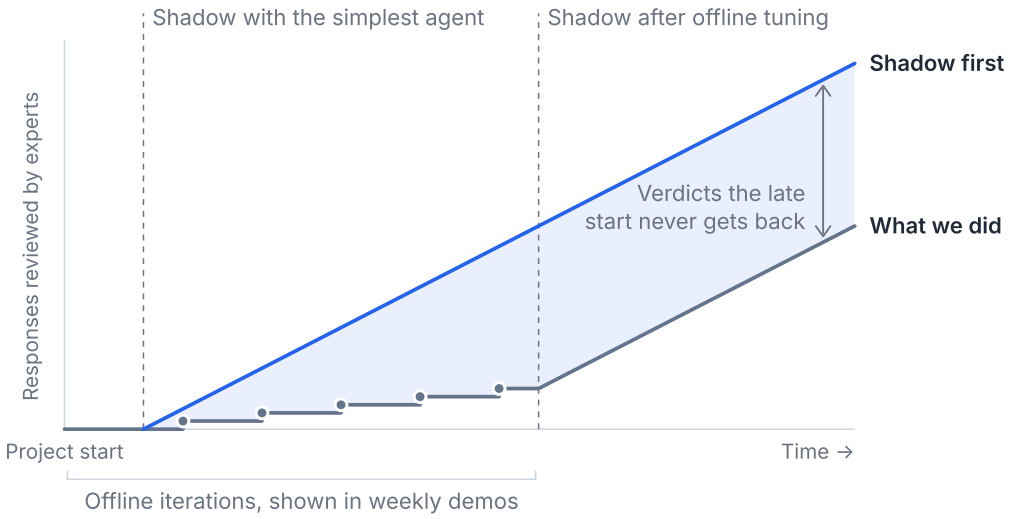
- Back to the opening question, with a frame for it.
- The theory of constraints starts from one observation: speed up anything that isn’t the bottleneck and nothing gets faster.
- So: where was ours, before and after coding agents?

With coding agents: the experts' time sets the pace



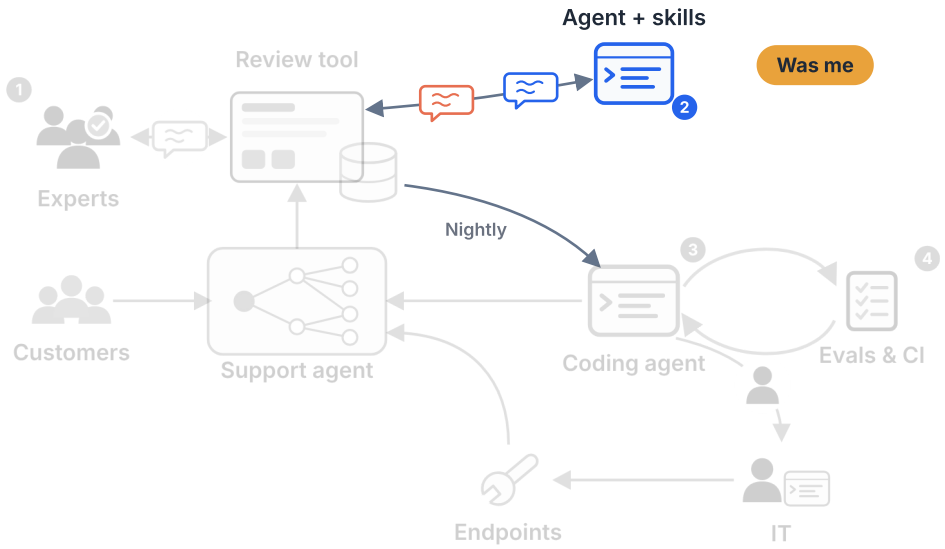
- Without a coding agent, I read the feedback, decided the fix and wrote the code; writing the code was the slow part, and the experts would have waited for me.
- With one, a fix I had specified was a branch, an eval case and a green build by the next morning, with little of my time.
- What limited progress was step 1: getting domain knowledge out of the experts' heads.
- That is what the missing test suite costs: we learned whether a reply was right only when an expert had time to read it.
- So the reviewers' time is a hard input when you estimate a project like this. It cost them less than it might have (they answered each ticket anyway), but it was one more item in a full day, and engagement and sponsorship can't be counted on. We were fortunate on both.

Get into shadow deployment sooner



- Schematic, not measured: the same reviewer capacity in both paths.
- We went live in shadow only once the offline numbers on historic tickets looked acceptable. That has it backwards.
- Before shadow, the experts saw only the handful of responses I demoed each week. The grades they could have given in those weeks are lost for good; the gap never closes.
- Replaying old tickets is the weaker signal anyway: a replayed ticket runs against today's records, not the customer's. Good for regression, poor for finding out what a good reply looks like.

The next bottleneck: who translates the feedback



- Step 2 was me: one engineer on the critical path of every improvement. Once the reviewers grade steadily, that is the next bottleneck.
- Handing step 2 to the coding agent on the raw feedback gave worse results than doing it myself; skill files and targeted guidelines narrowed the gap without closing it.
- The more interesting version changes who writes the skills. An agent with persistent memory, something like Hermes or Harness[2], could rewrite them from each batch of grades. The experts would be the only people left in the loop.
- A skill is only a prompt, not a guarantee.
- The risk is drift: the skills slowly stop matching how the business works. The only thing that would catch it is the experts' grading, which is already the bottleneck.
- I haven't solved this. I think it is solvable, with a better loop rather than more of my time.

Start at the bottleneck

1. **Settle what the loop should improve**, and what that would save.
2. **Build the review screen and shadow deployment first**, with the simplest system that can draft a reply behind it.
3. **Then grow the rules from the grades**, not from a whiteboard.

- Coding agents didn’t remove the hard part of this project; they made it obvious where it was.
- If I started a project of this shape again, this is the order.
- Lead into the closing line.

Until someone who knows the business reads the output every day, every design decision is a guess.

danielfridljand.de/post/the-missing-test-suite

- Thank you. Over to discussion.
- Prompts if it needs one: Where is the bottleneck in your projects now that code is cheap? Who in your projects plays the experts, and how do you get their time?
- Likely questions: How did you get the experts to engage? What approval rate did you reach? (Don't quote numbers the post doesn't.) An approval rate is not a cost reduction: the post on that. Why not let the agent handle everything from day one? How do you handle personal data in the review tool? Why build the review screen rather than use Langfuse's annotation queues? (Build the screen, not the storage behind it.)